Unidad 1 - Sistemas de almacenamiento de la información

Módulo: Gestión de base de datos

RA1: Reconoce los elementos de las bases de datos, analizando sus funciones y valorando la utilidad de los sistemas gestores.

Indice

- 1. Ficheros
  - 1.1 Ficheros planos (secuenciales)
  - 1.2 Ficheros indexados
    - Organización del índice: árboles vs. hashing
  - 1.3 Ficheros de acceso directo (aleatorio o relativo)
  - 1.4 Comparativa de tipos de ficheros
  - 1.5 Limitaciones de los sistemas de ficheros frente a las bases de datos
- 2. Bases de datos: conceptos, usos y tipos
  - 2.1 Concepto de base de datos
  - 2.2 Usos de las bases de datos
  - 2.3 Tipos de bases de datos según el modelo de datos
  - 2.4 Tipos de bases de datos según la ubicación de la información
- 3. Sistemas gestores de bases de datos (SGBD): funciones, componentes y tipos
  - 3.1 Concepto de SGBD
  - 3.2 Funciones de un SGBD
  - 3.3 Componentes (elementos) de un SGBD
    - 3.3.1 Diagrama 1: Visión general de componentes y flujos principales
    - 3.3.2 Diagrama 2: Flujo DML (consulta/actualización)
    - 3.3.3 Diagrama 3: Flujo DDL (definición de esquema)
    - 3.3.4 Diagrama 4: Flujo DCL (gestión de permisos y control)
    - 3.3.5 Diagrama 5: Flujo de transacciones, concurrencia y recuperación
  - 3.4 Tipos de sistemas gestores de bases de datos
- 4. Esquema-resumen
- 5. Glosario de términos clave

1. Ficheros

(CE-a: Se han analizado los distintos sistemas lógicos de almacenamiento y sus funciones)

- Un ordenador maneja y almacena una gran cantidad de información.
- La información se almacena en dispositivos como discos duros, pendrives, DVDs, etc.
- Esta información almacenada debe estar adecuadamente organizada.
- Para poder organizar la información se utilizan `los ficheros o archivos`

Antes de la aparición de las bases de datos, la información se almacenaba organizada en **ficheros** (archivos), gestionados directamente por los programas de aplicación o por el sistema operativo.

Un **fichero** es un conjunto de **registros**, y cada registro está formado por **campos**.

Según cómo se organizan y se accede a los registros dentro del fichero, distinguimos varios tipos:

1.1 Ficheros planos (secuenciales)

- Los registros se almacenan **uno detrás de otro**, en el orden en que se van grabando (o según un criterio de ordenación, como una clave).
- Para acceder a un registro concreto hay que **leer desde el principio** hasta encontrarlo (acceso secuencial).
    - **Ventajas:** estructura simple, fácil de crear y mantener, muy eficiente cuando se procesan todos los registros uno tras otro (por ejemplo, generar un listado completo).
    - **Inconvenientes:** acceso lento a un registro concreto cuando el fichero es grande, ya que hay que recorrerlo desde el inicio; las operaciones de inserción/borrado en medio del fichero son costosas
- **Ejemplo de uso:** ficheros de texto plano (`.txt`, `.csv`), copias de seguridad, procesos por lotes (*batch*).
- **Formato (texto o binario):** los ficheros planos se encuentran habitualmente en formato de texto legible (por ejemplo CSV), pero también pueden almacenarse en formato binario. En texto cada registro suele ser una línea y los campos se separan por delimitadores; en binario los registros pueden tener campos de tamaño fijo y ocupan menos espacio y se procesan más rápido. La elección depende de la necesidad de legibilidad/intercambio (texto) frente a rendimiento/compactación (binario).

Ejemplo práctico (.txt):

Imaginemos un fichero de texto plano llamado "clientes.txt" que almacena información de clientes, donde cada línea corresponde a un registro y los campos están separados por comas:

```
1,Garcia Perez,garcia@example.com,600123456
2,Lopez Martinez,lopez@example.com,600987654
3,Sanchez Diaz,sanchez@example.com,600555000
```

- Cada línea (por ejemplo `1,Garcia Perez,garcia@example.com,600123456`) es un **registro**: representa la información relacionada con una única entidad (aquí, un cliente).
- Los **campos** son las partes que componen el registro, separadas por comas en este ejemplo:
    - Campo 1: `1` (ID del cliente)
    - Campo 2: `Garcia Perez` (nombre)
    - Campo 3: `garcia@example.com` (correo electrónico)
    - Campo 4: `600123456` (teléfono)

> 📌 Nota: En ficheros planos los campos pueden separarse por comas (CSV), tabuladores, punto y coma u otros separadores, o bien tener longitudes fijas. Su simplicidad los hace útiles para intercambio de datos y pequeños procesos por lotes, pero su estructura no protege contra inconsistencias (por ejemplo, distinta ordenación de campos, falta de delimitadores o problemas con comas dentro de los campos). Para mitigar esto, es común usar comillas para campos de texto, incluir una cabecera con nombres de campos o aplicar reglas de validación en el proceso de importación. Si se requiere integridad, control de concurrencia y consultas complejas, conviene usar un SGBD.

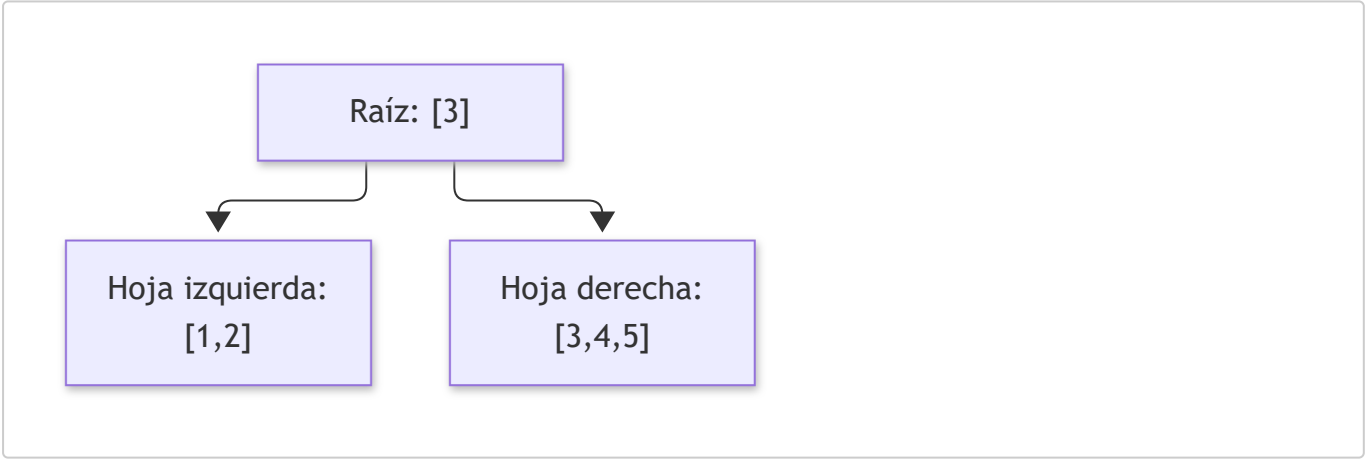
1.2 Ficheros indexados

- Además de los datos, se mantiene una **estructura de índice** (similar al índice de un libro) que asocia cada valor de una clave con la posición física del registro correspondiente en el fichero.
- Permiten **dos formas de acceso**: secuencial (recorriendo todos los registros) y **por clave** (a través del índice), combinando lo mejor de ambos mundos.
- El acceso mediante el índice es mucho más rápido que recorrer todo el fichero, ya que el índice suele organizarse en estructuras eficientes de búsqueda (por ejemplo, árboles B o B+).
    - **Inconvenientes:** ocupan más espacio (hay que guardar el índice además de los datos) y hay que mantener el índice actualizado cada vez que se modifica el fichero, lo que añade cierta sobrecarga en inserciones/borrados.

Organización del índice: árboles vs. hashing

Los índices pueden organizarse internamente con diferentes estructuras. Las dos más comunes son los árboles (p. ej. B-tree / B+tree) y las tablas hash. A continuación se explica brevemente qué conlleva cada opción y se muestra un diagrama conceptual.

- **Árboles (B-tree / B+tree):**
    - *Características:* mantienen las claves ordenadas, permiten búsquedas, inserciones y borrados en tiempo O(log m) donde m es el número de entradas del índice. Son particularmente útiles cuando se necesitan operaciones de rango (por ejemplo, "todas las claves entre A y B").
    - *Sobrecarga:* requieren mantener el equilibrio del árbol y pueden implicar operaciones de reestructuración (divisiones/fusiones de nodos) en inserciones y borrados.



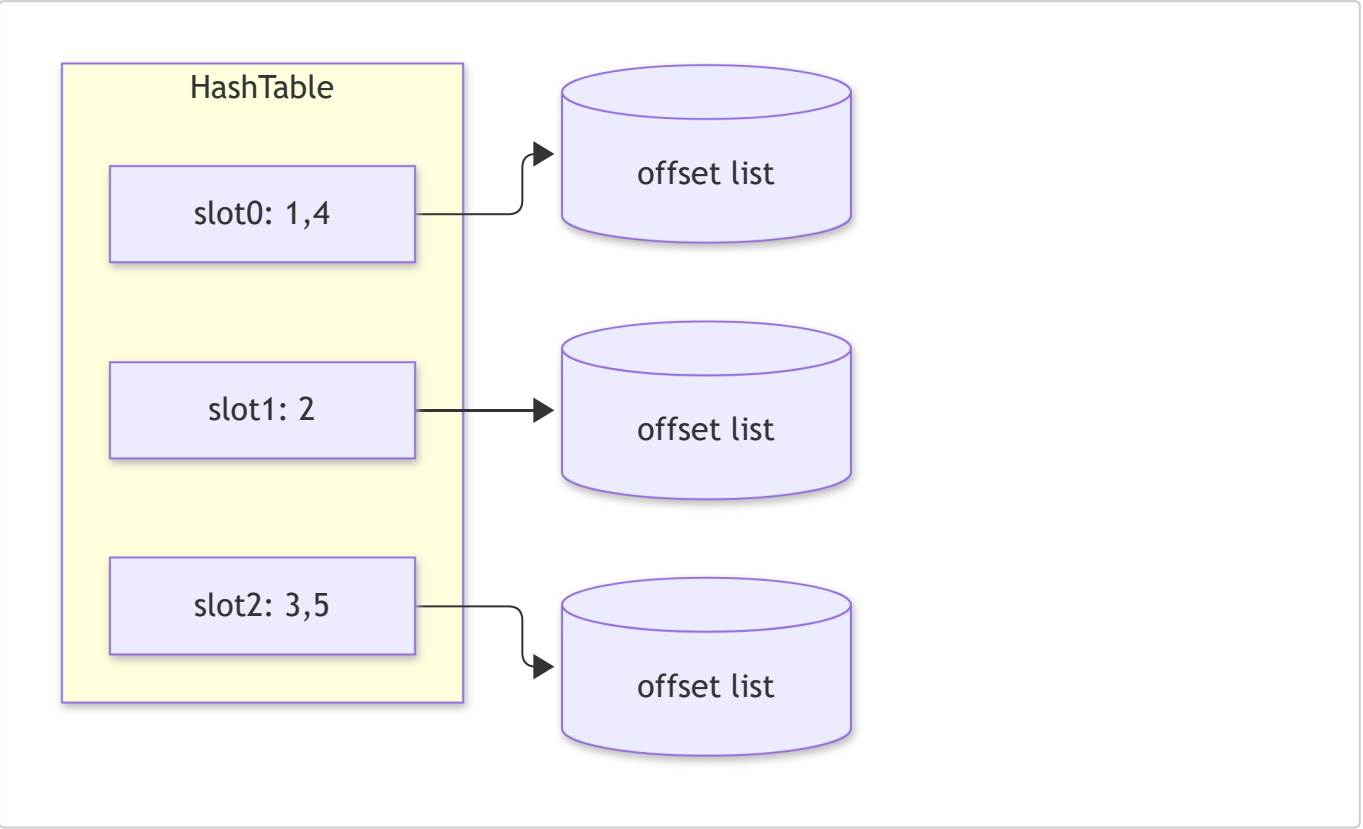
Ejemplo: `insertar claves 1..5 en un B-tree de orden t=2`

Explicación del ejemplo:

- En un B-tree (grado mínimo t = 2) cada nodo hoja puede contener como máximo 2t−1 = 3 claves y como mínimo t−1 = 1 clave. Eso explica el número «3» que aparece en la raíz: es la clave separadora (no significa que cada hoja deba tener 3 claves).
    - Insertar 1,2,3 en la hoja inicial → hoja = [1,2,3].
    - Insertar 4 provoca que la hoja exceda su capacidad (4 claves), se divide en dos hojas [1,2] y [3,4] y se promueve la clave 3 al nodo raíz.
    - Insertar 5 va al segmento derecho → hoja derecha pasa a [3,4,5] (dentro del máximo permitido).

Búsqueda de la clave 5 (en este árbol sería: comparar con la raíz [3], 5>3 → bajar por el puntero derecho y buscar en la hoja derecha.

- **Hashing (índice por dispersión):**
  - *Características:* una función hash convierte la clave en una posición (slot). La consulta suele ser O(1) en tiempo promedio para localizar la posición del registro. Es eficiente para búsquedas puntuales por clave.
  - *Sobrecarga:* debe gestionarse la resolución de colisiones (encadenamiento o sondeo) y no es eficiente para operaciones de rango o para mantener el orden de las claves.



`Explicación del ejemplo de hashing (claves 1..5):`

- Definimos una función hash simple: `h(key) = key mod 3` (3 slots: 0,1,2).
  - Nota: `mod` significa resto de la división entera. Por ejemplo `5 mod 3 = 2` porque 5 = 1*3 + 2.
- Insertamos claves 1,2,3,4,5 en ese orden:
  - h(1) = 1 → slot1: [1]
  - h(2) = 2 → slot2: [2]
  - h(3) = 0 → slot0: [3]
  - h(4) = 1 → slot1: [1,4] (colisión resuelta por encadenamiento)
  - h(5) = 2 → slot2: [2,5] (colisión resuelta por encadenamiento)
- Búsqueda de la clave 5:
  - calcular h(5)=2 → ir al `slot2`
  - recorrer la lista en `slot2` y localizar 5 (posible 1-2 comparaciones dependiendo de la posición)
- Operaciones típicas: 1 cálculo de hash + k comparaciones en la lista del slot (k es el número de elementos en el slot). En este ejemplo k=2, así que 1 cálculo + hasta 2 comparaciones.

Consecuencias:

Hashing ofrece localizaciones rápidas en promedio, pero el rendimiento depende de la función hash y de la carga por slot (factor de carga). El encadenamiento es sencillo y flexible.

En resumen: elegir árbol o hashing depende de los requisitos:

```
- búsquedas por rango y ordenadas → B-tree/B+tree
- búsquedas puntuales por clave con alta velocidad → hashing.
```

- **Formato (texto o binario):** los ficheros indexados suelen implementarse sobre formatos binarios que permiten direccionar con precisión offsets y manejar estructuras de índice (B-tree, B+tree) de forma eficiente. No obstante, también pueden existir soluciones híbridas: un fichero de datos en texto (CSV) con un índice externo binario que guarda offsets.

Ejemplo práctico (indexado):

Supongamos un fichero de datos `clientes.dat` donde cada registro ocupa una posición física (por ejemplo, número de registro) y existe un fichero índice `clientes.idx` que asocia la clave (ID) con esa posición:

Archivo de datos (clientes.dat) - `contenido conceptual`:

```
Registro#1: 1|Garcia Perez|garcia@example.com|600123456
Registro#2: 2|Lopez Martinez|lopez@example.com|600987654
Registro#3: 3|Sanchez Diaz|sanchez@example.com|600555000
```

Índice (clientes.idx) - entrada clave→posición:

```
1 -> Registro#1
2 -> Registro#2
3 -> Registro#3
```

- Si se busca el cliente con ID = 2, el sistema consulta el índice (muy rápido) y obtiene `Registro#2`, luego lee directamente la posición en el fichero de datos sin tener que recorrer todos los registros.
- En implementaciones reales el índice suele estar organizado en árboles (B-tree/B+tree) o en estructuras de hash, y puede residir en memoria para acelerar lecturas.

> 📌 Nota: Los ficheros indexados son especialmente útiles cuando las consultas por clave son frecuentes y se desea combinar acceso rápido por clave con la posibilidad de recorrer registros en orden.

1.3 Ficheros de acceso directo (aleatorio o relativo)

- Permiten acceder **directamente** a cualquier registro sin necesidad de recorrer los anteriores, normalmente calculando la posición física del registro a partir de su clave (por ejemplo, mediante una función *hash* o porque los registros tienen tamaño fijo y se calcula un desplazamiento).

- También se conocen como ficheros de **acceso aleatorio** o de **acceso relativo**.

    - **Ventajas:** acceso muy rápido a un registro concreto, independientemente de su posición en el fichero
    - **Inconvenientes:** es necesario conocer o calcular la posición del registro; no son eficientes para procesar todos los registros de forma ordenada; puede haber colisiones si dos claves generan la misma posición (en el caso de acceso por *hash*).

- **Formato (texto o binario):** los ficheros de acceso directo se implementan típicamente en formato binario con registros de tamaño fijo o estructuras que permitan calcular offsets fiables. Esto facilita el cálculo directo de posiciones y el acceso por hashing. Implementaciones que intentan usar texto para acceso directo existen (por ejemplo, manteniendo un mapa de offsets), pero suelen ser menos eficientes.

Ejemplo práctico (acceso directo mediante hashing):

Imaginemos un fichero `cuentas.dat` en el que queremos almacenar cuentas bancarias y acceder por número de cuenta. Usando hashing, definimos una función simple que convierte el número de cuenta en una posición (slot) del fichero:

- Supongamos tamaño del fichero = 1000 slots.
- Hash(numero_cuenta) mod 1000 = posición.

Ejemplo conceptual:

```
Hash(1000456) mod 1000 -> 456  => almacenar registro en slot 456
Hash(2000102) mod 1000 -> 102  => almacenar registro en slot 102
Hash(3000001) mod 1000 -> 1    => almacenar registro en slot 001
```

- Para buscar la cuenta 2000102 se calcula su hash y se accede directamente al slot 102, sin necesidad de leer otros registros.

- Si dos cuentas colisionan (mismo slot), el sistema debe resolver la colisión (por ejemplo, con encadenamiento -lista en ese slot- o con sondeo abierto).

Ejemplo práctico (acceso directo mediante desplazamiento fijo):

Si los registros tienen tamaño fijo (por ejemplo 200 bytes), y conocemos el número de registro n, la posición física en bytes se calcula como:

```
offset = (n - 1) * record_size
```

Así, para leer el registro nº 10 el sistema salta directamente a offset = 9 * 200 = 1800 bytes y lee 200 bytes.

> 📌 Nota: Los ficheros de acceso directo ofrecen lecturas muy rápidas para búsquedas puntuales, pero su diseño requiere planificación (tamaño de registro, estrategia de resolución de colisiones) y suelen usarse cuando la latencia de acceso es crítica.

1.4 Comparativa de tipos de ficheros

Tipo	Forma de acceso	Velocidad de acceso a un registro	Velocidad de proceso secuencial	Uso típico
Plano (secuencial)	Uno tras otro, desde el principio	Lenta	Muy rápida	Listados completos, copias de seguridad
Indexado	Secuencial o por índice	Rápida (vía índice)	Rápida	Sistemas con consultas frecuentes por clave
Acceso directo	Directa (cálculo de posición)	Muy rápida	Lenta/poco práctica	Localización puntual de registros concretos

1.5 Limitaciones de los sistemas de ficheros frente a las bases de datos

El uso de ficheros gestionados de forma independiente por cada aplicación presenta problemas:

- **Redundancia e inconsistencia de datos** (el mismo dato duplicado en varios ficheros).
- **Dificultad para compartir información** entre distintas aplicaciones.
- **Dependencia entre datos y programas** (cualquier cambio en la estructura obliga a modificar el programa).
- **Falta de control centralizado de la seguridad y de la integridad**.
- **Problemas de acceso concurrente** (varios usuarios/programas accediendo a la vez).

Estas limitaciones son las que justifican la aparición de las **bases de datos** y de los **sistemas gestores de bases de datos (SGBD)**.

---

2. Bases de datos: conceptos, usos y tipos

2.1 Concepto de base de datos

(CE-d: Se ha reconocido la utilidad de un sistema gestor de bases de datos)

Una **base de datos** es un conjunto de datos organizados, estructurados y relacionados entre sí, almacenados de forma conjunta, sin redundancias innecesarias, y diseñados para ser utilizados por diferentes usuarios y aplicaciones de forma simultánea.

A diferencia de un simple conjunto de ficheros, una base de datos:

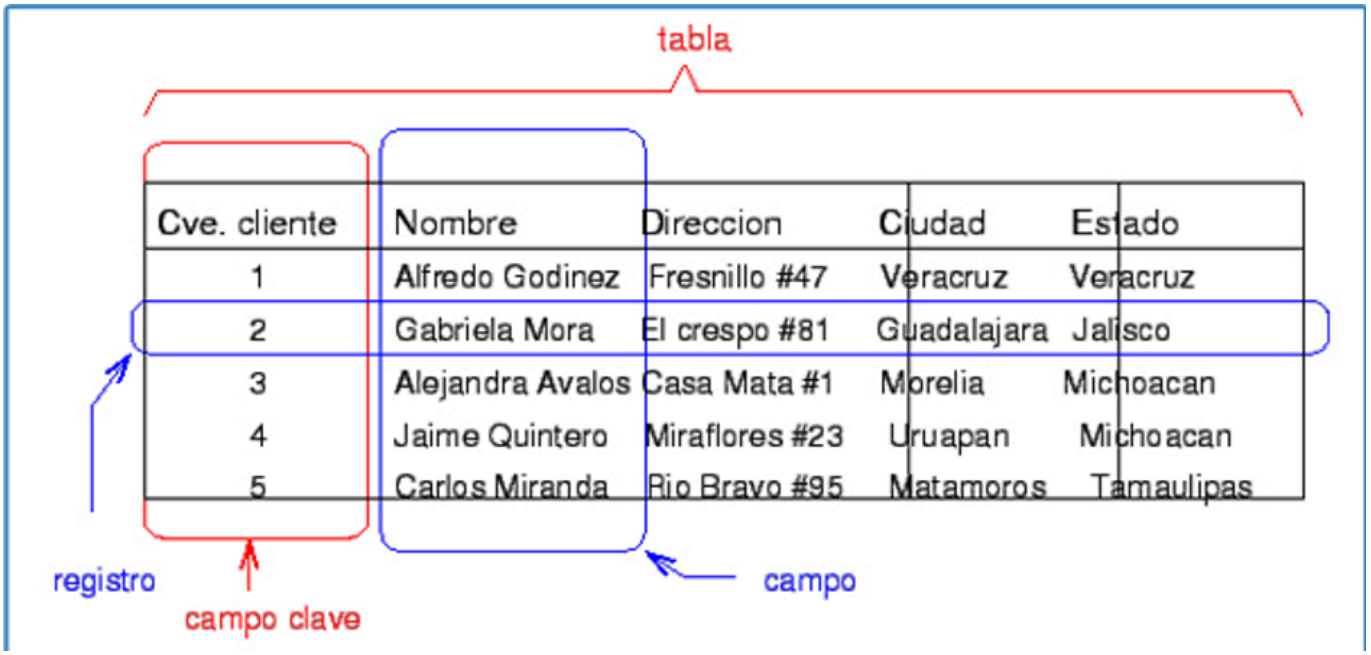
- Está gestionada por un software específico (el **SGBD**).
- Los datos son **independientes** de los programas que los usan.
- Permite compartir la información entre varios usuarios y aplicaciones a la vez.
- Garantiza la **integridad**, **consistencia** y **seguridad** de los datos.

Una base de datos se organiza en **tablas**, **colecciones**, **objetos** (depende del modelo de BD) que se relacionan entre si para que la información esté almacenada de forma ordenada y coherente.

Conceptos básicos:

- **Dato:** Es una información concreta sobre algo y se caracteriza por pertenecer a un tipo. Por ejemplo 2026 es un dato que representa el año actual y su tipo es un número entero.

- **Tipo de dato:** Indica la naturaleza del dato. Representa que conjunto de valores puede tomar. Son tipos de datos texto, carácter, numérico entero, numérico real, fecha, etc.
- **Campo:** Es un representación de un conjunto de datos. Por ejemplo el campo para representar la fecha de nacimiento de los alumnos puede llamarse FechaNac.
- **Registro:** Es un conjunto de datos referentes a un mismo elemento. Por ejemplo, un alumno concreto tendría varios datos como dni, nombre, apellidos, fecha de nacimiento, etc.
- **Tabla:** Es un conjunto de registros representado con un nombre que contiene toda la información de una parte del sistema de información. Una `base de datos relacional` se organiza en `tablas`. Por ejemplo, una base de datos de un centro de estudios podría tener las tablas profesores, alumnos, módulos, matriculaciones, etc.



- **Consulta:** Se trata de una instrucción para hacer peticiones de datos a una base de datos para que se haga una búsqueda en la base de datos de los registros que cumplen con las condiciones expresadas en la instrucción.
- **Índice:** Es una estructura que almacena los campos clave de una tabla para que sea más rápido encontrar y ordenar los registros de la tabla.
- **Clave primaria (PK):** Identificador único de cada registro; implícitamente suele tener un índice único. Explicación: la PK garantiza unicidad y el motor crea un índice para localizar filas por su valor rápidamente.
- **Clave foránea (FK):** Valor que enlaza a la PK de otra tabla; no siempre crea índice automáticamente. Explicación: la FK mantiene la integridad referencial; conviene crear un índice en la columna FK para acelerar joins y comprobaciones de integridad.
- **Vista:** Es una transformación que se hace de una o más tablas para obtener una tabla que será visible para determinados usuarios. Esta tabla es virtual, no permanece almacenada.
- **Informe:** Es un documento que se genera como resultado de una consulta a la base de datos y que es fácilmente legible para los usuarios.
- **Guiones o scripts:** Son conjuntos de instrucciones que realizan operaciones avanzadas de mantenimiento de los datos.

2.2 Usos de las bases de datos

(CE-d: Se ha reconocido la utilidad de un sistema gestor de bases de datos)

Las bases de datos se utilizan prácticamente en cualquier ámbito donde haya que gestionar información de forma organizada:

- **Gestión empresarial:** clientes, proveedores, facturación, nóminas, inventario.
- **Administración pública:** censo, historiales médicos, registros civiles.
- **Comercio electrónico:** catálogos de productos, pedidos, usuarios.
- **Redes sociales y aplicaciones web:** perfiles de usuario, publicaciones, mensajes.
- **Sistemas científicos y de investigación:** almacenamiento de grandes volúmenes de datos experimentales.
- **Aplicaciones móviles y de escritorio:** almacenamiento local de configuración o datos del usuario.

2.3 Tipos de bases de datos según el modelo de datos

(CE-b: Se han identificado los distintos tipos de bases de datos según el modelo de datos utilizado)

El **modelo de datos** define cómo se organizan, estructuran y relacionan los datos dentro de la base de datos.

Modelo	Descripción	Ejemplos de SGBD
Jerárquico	Los datos se organizan en forma de árbol (relaciones padre-hijo, 1:N). Un hijo solo puede tener un padre.	IMS (IBM)
En red	Similar al jerárquico, pero un registro puede tener varios "padres" (relaciones N:M), formando una estructura en forma de grafo.	IDS, IDMS
Relacional	Los datos se organizan en **tablas** (filas o tuplas y columnas o atributos), relacionadas mediante claves. Es el modelo más utilizado en la actualidad.	MySQL, PostgreSQL, Oracle, SQL Server
Orientado a objetos	Los datos se representan como objetos con atributos y métodos, de forma similar a la programación orientada a objetos.	db4o, ObjectDB
Objeto-relacional	Modelo híbrido: es relacional, pero admite tipos de datos complejos y objetos.	PostgreSQL, Oracle
NoSQL (no relacional)	Modelos alternativos al relacional pensados para grandes volúmenes de datos y alta escalabilidad: documentales, clave-valor, basados en grafos o en columnas.	MongoDB (documental), Redis (clave-valor), Neo4j (grafos), Cassandra (columnas)

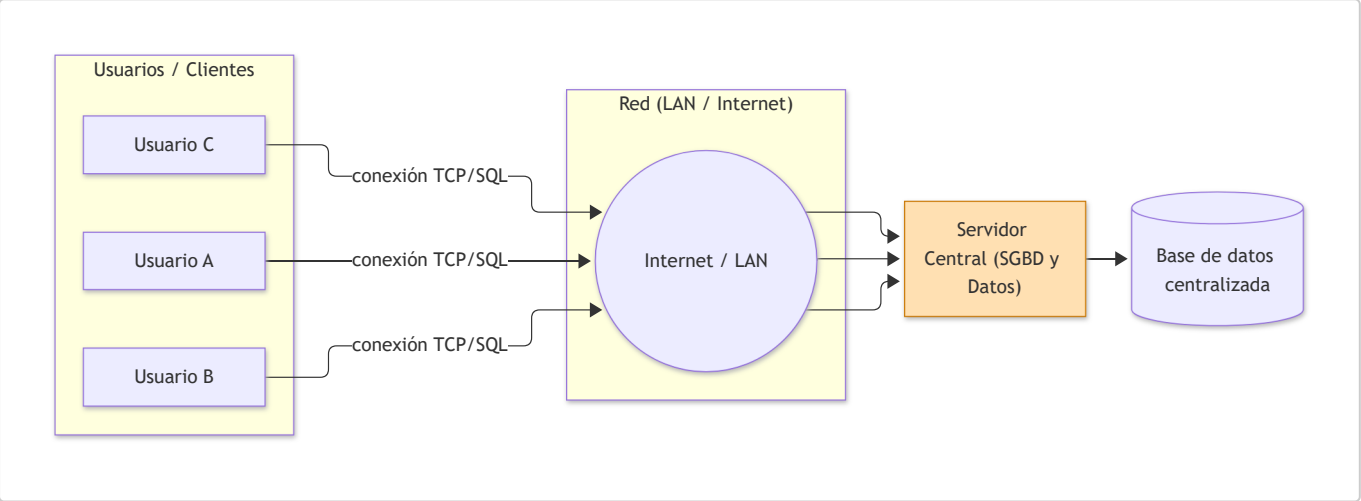
> **Evolución histórica:** Jerárquico → En red → Relacional → Orientado a objetos / NoSQL. El modelo **relacional**, propuesto por E. F. Codd en 1970, sigue siendo el más extendido, aunque el modelo **NoSQL** ha ganado peso con el Big Data y las aplicaciones web de gran escala.

2.4 Tipos de bases de datos según la ubicación de la información

(CE-c: Se han identificado los distintos tipos de bases de datos en función de la ubicación de la información)

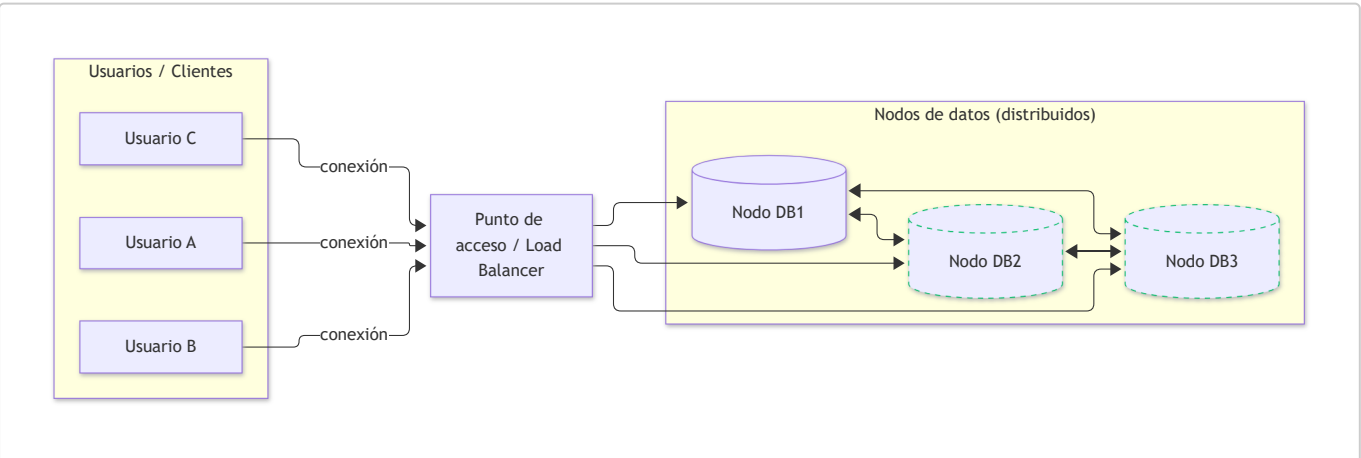
Tipo	Descripción	Ventajas	Inconvenientes
Centralizada	Toda la información se almacena en un único lugar físico (un servidor o equipo).	Fácil de administrar, control y seguridad centralizados, menor coste inicial	Punto único de fallo, posibles cuellos de botella con muchos usuarios
Distribuida	La información está repartida en varios equipos o ubicaciones físicas distintas, aunque el usuario la percibe como una única base de datos.	Mayor disponibilidad y tolerancia a fallos, mejor rendimiento, escalabilidad	Mayor complejidad de diseño y administración, problemas de sincronización entre nodos

Modelo centralizado: todos los datos y la lógica residen en un único servidor (SGBD). Los usuarios se conectan al mismo punto para consultar y modificar datos; la administración, backups y seguridad son centralizados.



Modelo distribuido: los usuarios se conectan a un punto de entrada (p. ej. balanceador), que reparte las solicitudes entre varios nodos de datos. Los nodos pueden replicarse entre sí o almacenar fragmentos diferentes.

- *Ventajas:* alta disponibilidad y escalabilidad.
- *Inconvenientes:* mayor complejidad en sincronización y administración.



Dentro de las bases de datos distribuidas se suelen distinguir además:

- **Replicadas:** existen copias idénticas de los datos en distintas ubicaciones (mejora la disponibilidad y el rendimiento de lectura).
- **Fragmentadas:** los datos se dividen en partes (fragmentos) que se reparten entre distintos nodos (cada nodo almacena solo una porción de los datos).
- **En la nube:** alojadas en servidores de un proveedor externo (AWS, Azure, Google Cloud), accesibles a través de Internet.

---

3. Sistemas gestores de bases de datos (SGBD): funciones, componentes y tipos

3.1 Concepto de SGBD

(CE-d: Se ha reconocido la utilidad de un sistema gestor de bases de datos)

Un **Sistema Gestor de Bases de Datos (SGBD)**, en inglés *DBMS (Database Management System)*, es el software que permite **crear, definir, manipular y administrar** una base de datos, actuando como intermediario entre los usuarios/aplicaciones y los datos almacenados físicamente.

3.2 Funciones de un SGBD

(CE-e: Se ha descrito la función de cada uno de los elementos de un sistema gestor de bases de datos)

- **Definición de la estructura de los datos:** permite crear tablas, especificar tipos de datos, relaciones y restricciones.
- **Manipulación de los datos:** inserción, consulta, actualización y eliminación de datos.
- **Independencia de los datos:** separa la estructura lógica de los datos de su almacenamiento físico, de modo que los programas no dependen de cómo se guardan realmente.
- **Reducción de la redundancia:** evita duplicar la misma información en distintos lugares.
- **Garantía de integridad y consistencia:** aplica reglas y restricciones para que los datos sean correctos y coherentes.
- **Control de acceso y seguridad:** define qué usuarios pueden acceder a qué datos y con qué permisos.
- **Control de la concurrencia:** gestiona el acceso simultáneo de múltiples usuarios sin conflictos ni pérdidas de información.
- **Gestión de transacciones:** garantiza que las operaciones se ejecuten de forma completa y correcta (propiedades **ACID**: Atomicidad, Consistencia, Aislamiento y Durabilidad).
- **Copias de seguridad y recuperación:** permite realizar *backups* y restaurar la base de datos ante fallos o pérdidas de datos.
- **Optimización de consultas:** determina la forma más eficiente de ejecutar cada consulta (uso de índices, orden de las operaciones).

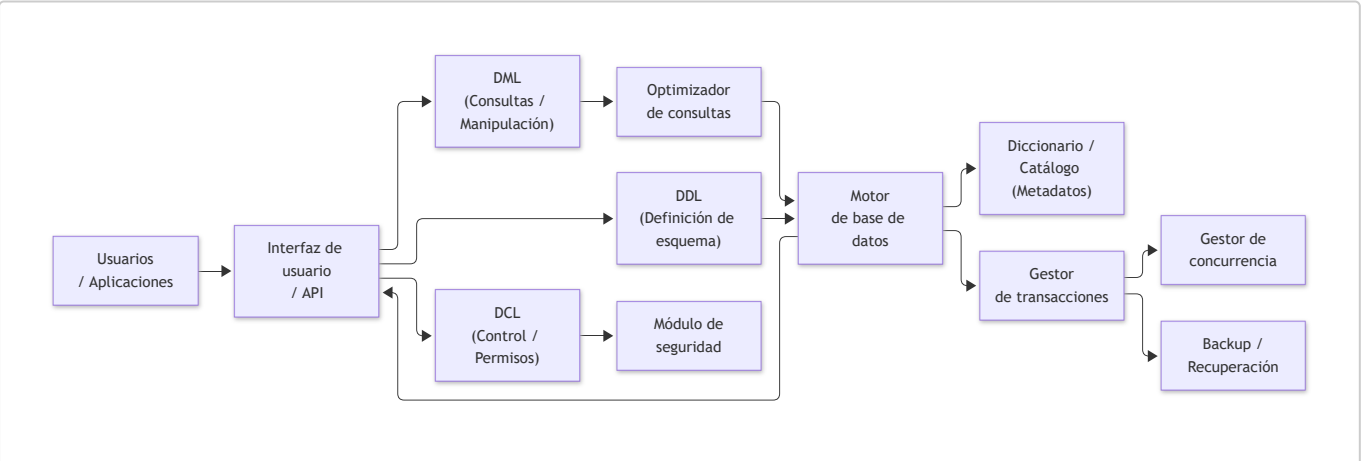
3.3 Componentes (elementos) de un SGBD

(CE-e: Se ha descrito la función de cada uno de los elementos de un sistema gestor de bases de datos)

Componente	Función
Motor de base de datos	Núcleo del sistema; se encarga del almacenamiento, recuperación y actualización física de los datos.

Componente	Función
Lenguaje de Definición de Datos (DDL)	Permite definir la estructura de la base de datos: tablas, tipos de datos, relaciones, restricciones (p. ej. `CREATE TABLE`).
Lenguaje de Manipulación de Datos (DML)	Permite consultar y modificar los datos: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
Lenguaje de Control de Datos (DCL)	Gestiona permisos y seguridad de acceso (`GRANT`, `REVOKE`).
Diccionario de datos (catálogo del sistema)	Almacena los **metadatos**: información sobre tablas, columnas, tipos, relaciones, usuarios y permisos.
Gestor de transacciones	Garantiza el cumplimiento de las propiedades ACID en cada operación.
Gestor de la concurrencia	Controla el acceso simultáneo de varios usuarios, evitando bloqueos e interbloqueos.
Módulo de seguridad y autorización	Controla la autenticación de usuarios y sus permisos sobre los datos.
Módulo de copia de seguridad y recuperación	Permite realizar copias de seguridad y restaurar la base de datos tras un fallo.
Optimizador de consultas	Analiza las consultas y decide la estrategia de ejecución más eficiente.
Interfaz de usuario / API	Permite a los usuarios y aplicaciones interactuar con el SGBD (línea de comandos, interfaz gráfica, conectores ODBC/JDBC).

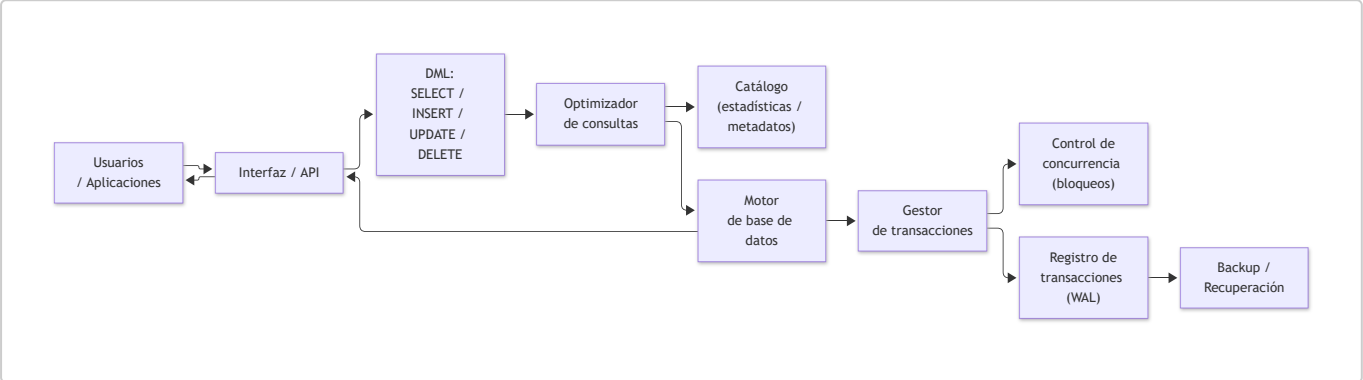
3.3.1 Diagrama 1: Visión general de componentes y flujos principales



- El usuario o la aplicación envía solicitudes a través de la interfaz (UI / API).
- Desde la interfaz se lanzan tres tipos de acciones: operaciones sobre los datos (DML), definiciones de esquema (DDL) y órdenes de control/permiso (DCL).
- Las peticiones DML pasan por el optimizador de consultas y llegan al motor de base de datos para su ejecución física.
- Las sentencias DDL se envían al motor para aplicar cambios en el esquema (creación/alteración/eliminación de objetos).

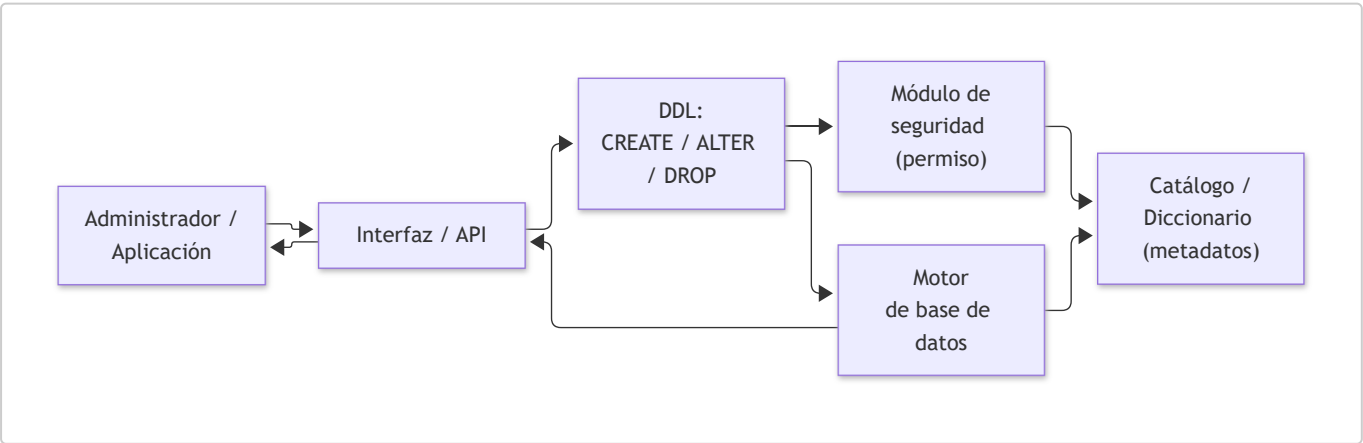
- Las órdenes DCL se dirigen al módulo de seguridad para comprobar/gestionar permisos.
- El motor accede al diccionario de datos / catálogo para leer metadatos y estadísticas, y coordina las operaciones mediante el gestor de transacciones.
- El gestor de transacciones actúa sobre el control de concurrencia y los mecanismos de backup/recuperación.
- Finalmente el motor devuelve resultados y estado a la interfaz para que lleguen al usuario.

3.3.2 Diagrama 2: Flujo DML (consulta/actualización)



- El usuario envía una consulta o modificación (SELECT/INSERT/UPDATE/DELETE) a través de la interfaz.
- La petición DML llega al optimizador de consultas, que consulta el catálogo (estadísticas, índices y metadatos) para elegir un plan eficiente.
- El optimizador entrega el plan al motor de base de datos, que ejecuta las operaciones físicas sobre los datos.
- Durante la ejecución, el motor interactúa con el gestor de transacciones para asegurar propiedades ACID: registra las operaciones (WAL / logs) y coordina bloqueos con el control de concurrencia.
- Los logs pueden usarse posteriormente por el subsistema de backup/recuperación para restaurar el estado en caso de fallo.
- El motor devuelve el resultado a la interfaz y de ahí al usuario.

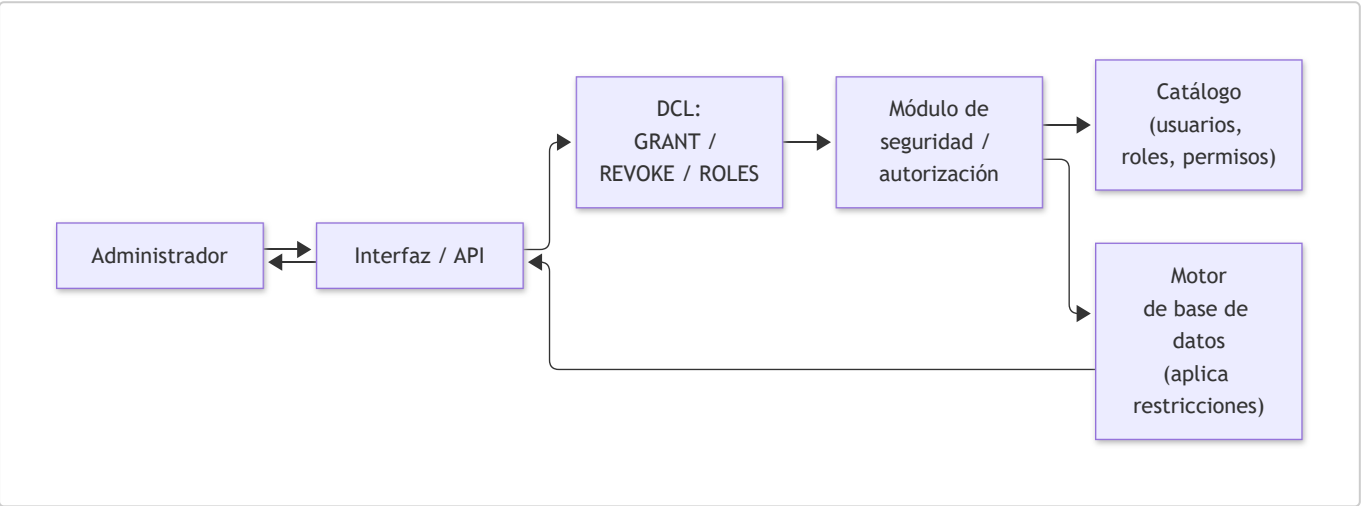
3.3.3 Diagrama 3: Flujo DDL (definición de esquema)



- Un administrador o aplicación envía una sentencia DDL (CREATE / ALTER / DROP) mediante la interfaz.

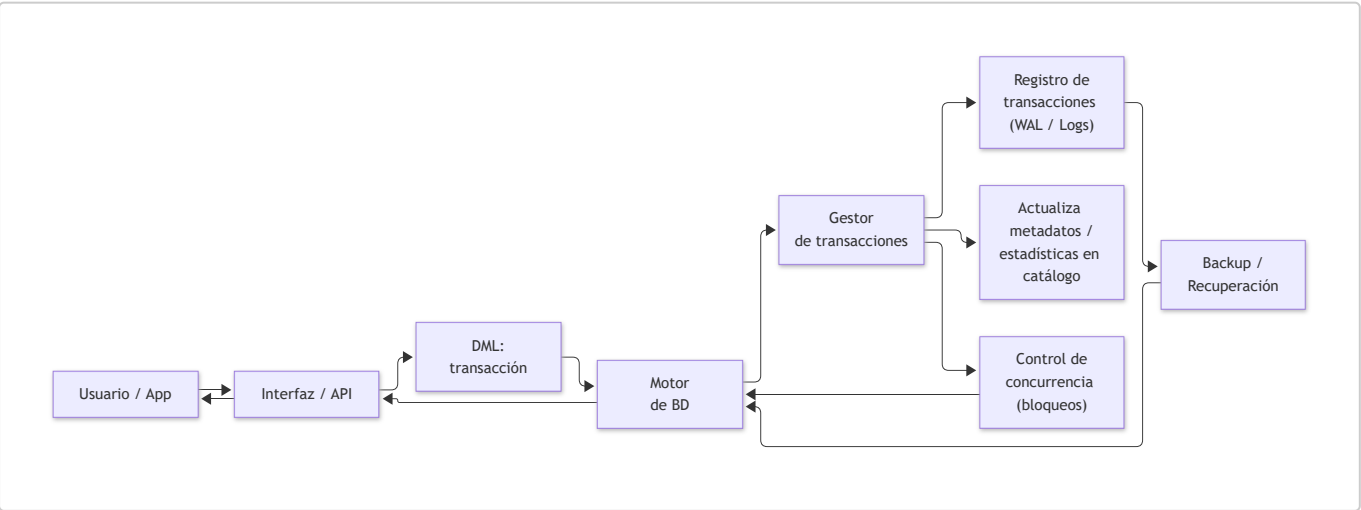
- Antes de aplicar cambios críticos, el DDL puede pasar por el módulo de seguridad para verificar permisos.
- El motor de base de datos procesa la sentencia DDL y actualiza la estructura física y lógica de la base de datos.
- El motor actualiza el catálogo/diccionario de datos con la nueva información del esquema (tablas, columnas, restricciones, etc.).
- El motor comunica el resultado de la operación a la interfaz y por tanto al administrador.

3.3.4 Diagrama 4: Flujo DCL (gestión de permisos y control)



- Un administrador emite comandos DCL (por ejemplo GRANT o REVOKE) desde la interfaz.
- El DCL se gestiona en el módulo de seguridad/autorización que administra usuarios, roles y privilegios.
- El módulo de seguridad consulta y actualiza el catálogo donde se almacenan los metadatos de usuarios/roles/permisos.
- Eventualmente el módulo de seguridad informa al motor para que aplique o haga cumplir las restricciones sobre operaciones futuras.
- El resultado del cambio (éxito / fallo) se comunica de vuelta a la interfaz y al administrador.

3.3.5 Diagrama 5: Flujo de transacciones, concurrencia y recuperación



- El usuario inicia una transacción (conjunto de operaciones DML) a través de la interfaz.
- La transacción es procesada por el motor de BD y coordinada por el gestor de transacciones.
- El gestor de transacciones usa el control de concurrencia para gestionar bloqueos y aislamientos entre transacciones concurrentes, evitando inconsistencias y conflictos.
- Simultáneamente, el gestor de transacciones escribe registros en el log (WAL / transaction log) para garantizar durabilidad.
- Los registros sirven para que el módulo de backup/recuperación pueda restaurar la base de datos hasta un estado consistente tras un fallo.
- Durante la ejecución la transacción puede actualizar metadatos y estadísticas en el catálogo.
- Cuando la transacción termina (commit o rollback), el motor devuelve el resultado a la interfaz y al usuario.

3.4 Tipos de sistemas gestores de bases de datos

(CE-f: Se han clasificado los sistemas gestores de bases de datos)

Los SGBD se pueden clasificar según varios criterios:

- **Según el modelo de datos que implementan:** jerárquicos, en red, relacionales (MySQL, PostgreSQL, Oracle, SQL Server), orientados a objetos, objeto-relacionales y NoSQL (documentales, clave-valor, columnares, grafos).
- **Según el número de usuarios que soportan:**
  - **Monousuario:** solo un usuario puede acceder a la vez (p. ej. Microsoft Access en modo local).
  - **Multiusuario:** varios usuarios pueden acceder simultáneamente (p. ej. MySQL, Oracle, SQL Server).
- **Según la ubicación de los datos que gestionan:** SGBD centralizados y SGBD distribuidos.
- **Según la licencia:**
  - **Libres / de código abierto:** MySQL (Community), MariaDB, PostgreSQL, SQLite.
  - **Propietarios / comerciales:** Oracle Database, Microsoft SQL Server, IBM Db2.
- **Según el tamaño y ámbito de uso:**
  - **De escritorio / embebidos:** SQLite, Microsoft Access (pequeñas aplicaciones, poco volumen de datos).
  - **Empresariales / de gran capacidad:** Oracle, SQL Server, PostgreSQL, MySQL, Db2 (grandes volúmenes, alta concurrencia).

---

4. Esquema-resumen

```
Sistemas de almacenamiento de la información
│
├── 1. FICHEROS
│     ├── Planos (secuenciales)
│     ├── Indexados
│     └── De acceso directo (aleatorio/relativo)
│
```

```
├── 2. BASES DE DATOS
│      ├── Concepto y usos
│      ├── Según el MODELO DE DATOS
│      │     Jerárquico · En red · Relacional · Orientado a objetos ·
│      │     Objeto-relacional · NoSQL
│      └── Según la UBICACIÓN
│            Centralizada · Distribuida (replicada, fragmentada, en la nube)
│
└── 3. SGBD (Sistema Gestor de Bases de Datos)
       ├── Funciones: independencia, integridad, seguridad, concurrencia,
       │              transacciones (ACID), backup/recuperación, optimización
       ├── Componentes: motor, DDL, DML, DCL, diccionario de datos,
       │                gestor de transacciones, gestor de concurrencia,
       │                seguridad, backup/recovery, optimizador, interfaz
       └── Tipos: por modelo de datos, por nº de usuarios, por ubicación,
                  por licencia, por tamaño/ámbito de uso
```

---

5. Glosario de términos clave

- **Registro:** conjunto de campos que forman una unidad de información dentro de un fichero o tabla.
- **Campo:** cada uno de los datos elementales que componen un registro.
- **Índice:** estructura auxiliar que agiliza la búsqueda de registros por una clave.
- **SGBD/DBMS:** Sistema Gestor de Bases de Datos.
- **Metadatos:** datos que describen la estructura de otros datos.
- **Clave primaria (PK):** campo que identifica de forma única cada registro.
- **Clave ajena/foránea (FK):** campo que referencia la clave primaria de otra tabla.
- **Transacción:** conjunto de operaciones que se ejecutan como una unidad indivisible.
- **ACID:** Atomicidad, Consistencia, Aislamiento (Isolation) y Durabilidad.
- **Redundancia:** repetición innecesaria de datos.
- **Concurrencia:** acceso simultáneo de varios usuarios a los mismos datos.
